



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 16

Control inteligente de iluminación mediante Bluetooth usando Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Clave:

Docente:

Ing. Jesús Padrón

Fecha:

14 de Mayo de 2026

Índice

1. Introducción	2
1.1. Objetivos	2
1.2. Trabajo Previo	3
1.2.1. El relevador electromecánico	3
1.2.2. El transistor NPN como interruptor	3
1.2.3. El diodo flyback	3
1.2.4. La fotorresistencia (LDR)	3
2. Desarrollo de la Práctica	4
2.1. Diseño del Circuito	4
2.2. Estructura del Proyecto Firmware	4
2.3. Descripción de los Archivos de Código	5
2.3.1. Archivo <code>lampara.gatt</code> – Servicio BLE personalizado	5
2.3.2. Archivo <code>btstack_config.h</code> – Configuración de BTstack	5
2.3.3. Archivo <code>main.c</code> – Lógica principal del firmware	6
2.3.4. Histéresis en el modo automático	6
2.4. Diseño de la Aplicación Móvil en Flutter	7
2.4.1. Estructura de la aplicación	7
2.4.2. Comunicación BLE en Flutter	7
2.5. Proceso de Compilación	8
2.6. Montaje y Pruebas	8
2.7. Complicaciones y Soluciones	10
3. Preguntas de Repaso	10
4. Conclusiones	15
Anexos	16
Evidencia Fotográfica	26

1. Introducción

En la presente práctica se diseña e implementa un sistema embebido de **control inteligente de iluminación** basado en la **Raspberry Pi Pico W**, que permite encender y apagar una carga eléctrica (bombillo de 120 VCA) de manera **inalámbrica** mediante el protocolo **Bluetooth Low Energy (BLE)**. El sistema integra tres modos de operación: *manual*, *automático por luz ambiente* y *temporizado*, controlados desde una aplicación móvil desarrollada en **Flutter**.

Desde el punto de vista del hardware, la práctica reúne varios conceptos fundamentales de la electrónica embebida: el aislamiento de cargas mediante un **relevador electromecánico de 5 V**, la amplificación de corriente mediante un **transistor NPN 2N2222A** configurado como interruptor electrónico, la protección contra picos inductivos mediante un **diodo de retorno (flyback) 1N4007**, y la lectura analógica del ambiente mediante una **fotorresistencia (LDR)** conectada al ADC del RP2040.

Desde el punto de vista del software, se aprovecha la biblioteca **BTstack** de BlueKitchen para implementar un servidor BLE personalizado con un servicio GATT propietario que expone una característica de control con permisos de lectura y escritura, sobre la cual la aplicación móvil envía comandos textuales (M:0, M:1, A:, T:30) que el firmware interpreta para cambiar el modo de operación o el tiempo del temporizador.

La importancia de este tipo de sistemas radica en su aplicación directa a la domótica, la automatización industrial y el ahorro energético. La lógica de encendido por sensor de luz emula el comportamiento de las lámparas de alumbrado público modernas, mientras que el temporizador y el control manual remoto son funciones comunes en cualquier sistema de **smart home** comercial.

1.1. Objetivos

- Implementar un sistema de encendido y apagado inalámbrico mediante Bluetooth LE utilizando la Raspberry Pi Pico W.
- Utilizar un relevador controlado mediante transistor NPN 2N2222A para manejar una carga eléctrica externa.
- Diseñar una aplicación móvil en Flutter que permita controlar el sistema desde un dispositivo Android.
- Implementar un temporizador configurable desde la aplicación móvil con cuenta regresiva visible.
- Implementar un modo automático de activación utilizando una fotorresistencia (LDR) leída por el ADC del RP2040.
- Comprender los principios de aislamiento de cargas, protección contra picos inductivos y amplificación de corriente en electrónica de potencia.

1.2. Trabajo Previo

1.2.1. El relevador electromecánico

Un **relevador** (o *relé*) es un interruptor accionado por una bobina electromagnética. Cuando se aplica corriente a la bobina, el campo magnético generado atrae una armadura metálica que cierra o abre contactos mecánicos, permitiendo controlar circuitos de alta potencia mediante señales de baja potencia. El modelo de 5 V utilizado en esta práctica consume aproximadamente 70 mA a través de su bobina, una corriente muy superior a los 12 mA máximos que puede suministrar un pin GPIO de la Pico W (que opera a 3.3 V con un máximo recomendado de 8 mA).

Esta diferencia hace **imprescindible** el uso de una etapa de amplificación de corriente. El relé también introduce *aislamiento galvánico* entre el lado de control (microcontrolador, 3.3-5 V) y el lado de carga (red eléctrica, 120 VCA), protegiendo al sistema lógico de transitorios peligrosos.

1.2.2. El transistor NPN como interruptor

El **2N2222A** es un transistor bipolar NPN de propósito general capaz de conmutar hasta 800 mA con tensiones de hasta 40 V. Configurado en *modo saturación*, opera como un interruptor controlado por corriente: una pequeña corriente en la base (I_B) permite el paso de una corriente mucho mayor entre colector y emisor ($I_C = h_{FE} \cdot I_B$). Con $h_{FE} \approx 100$ y una resistencia de base de 1 k Ω alimentada por un GPIO de 3.3 V, se obtiene:

$$I_B = \frac{3,3 - 0,7}{1000} = 2,6 \text{ mA}, \quad I_C \approx 260 \text{ mA}$$

Más que suficiente para activar la bobina del relevador. Esto significa que el GPIO solamente entrega 2.6 mA, muy por debajo del límite seguro.

1.2.3. El diodo flyback

Cuando la bobina del relé se desenergiza, el colapso súbito del campo magnético induce un **pico de tensión inverso** (back-EMF) que puede alcanzar varios cientos de voltios, suficiente para destruir el transistor. El **diodo 1N4007** conectado en *antiparalelo* con la bobina proporciona un camino de baja impedancia por donde la energía almacenada se disipa de forma segura. Este diodo se denomina *flyback*, *freewheeling* o *snubber* y es indispensable en cualquier circuito que conmute cargas inductivas.

1.2.4. La fotorresistencia (LDR)

Una **LDR** (*Light Dependent Resistor*) es un componente pasivo construido con un semiconductor sensible a la luz, típicamente **sulfuro de cadmio (CdS)**. Su resistencia varía **inversamente proporcional** a la intensidad luminosa incidente: en oscuridad total puede superar 1 M Ω , mientras que bajo iluminación directa cae a unos pocos cientos de ohms. Conectada en un divisor de tensión con una resistencia fija (10 k Ω),

genera un voltaje analógico que el ADC de 12 bits del RP2040 lee como un valor entre 0 y 4095.

2. Desarrollo de la Práctica

2.1. Diseño del Circuito

El circuito completo se compone de tres bloques funcionales:

1. **Etapas de sensado:** la LDR se conecta entre 3.3 V y el pin GP26 (ADC0), formando un divisor de tensión con una resistencia de 10 k Ω a tierra. La tensión leída por el ADC es proporcional a la corriente que atraviesa la LDR, y por tanto a la luz incidente.
2. **Etapas de amplificación:** el pin GP15 alimenta la base del transistor 2N2222A a través de una resistencia limitadora de 1 k Ω . El emisor va a tierra y el colector a la bobina del relé (terminal negativo), cuyo terminal positivo se conecta a 5 V.
3. **Etapas de protección y carga:** el diodo 1N4007 se conecta en antiparalelo con la bobina del relé (cátodo a 5 V, ánodo al colector del transistor). Los contactos del relé conectan el bombillo de 120 VCA a la red eléctrica.

Cuadro 1: Conexiones del circuito

Componente	Pin Pico W	Función
LDR (divisor con 10 k Ω)	GP26 (ADC0)	Lectura analógica de luz
Base 2N2222A (vía R=1 k Ω)	GP15	Control del relevador
LED indicador externo	GP2	Estado visible local
LED del módulo CYW43	WL_GPIO	Estado interno BLE
VBUS / VSYS	5 V	Alimentación del relé
GND	GND	Referencia común

2.2. Estructura del Proyecto Firmware

El proyecto del lado del microcontrolador se compila en un único ejecutable `lampara_ble.uf2` a partir de los siguientes archivos:

Cuadro 2: Archivos del firmware y su función

Archivo	Función
<code>main.c</code>	Punto de entrada, inicialización de hardware, máquina de estados de modos y manejadores BLE
<code>lampara.gatt</code>	Definición declarativa del servicio GATT personalizado y la característica de control
<code>btstack_config.h</code>	Configuración estática de la pila BTstack: roles, buffers, conexiones
<code>CMakeLists.txt</code>	Configuración del sistema de compilación: librerías, includes y generación del archivo <code>.uf2</code>
<code>pico_sdk_import.cmake</code>	Localización del Pico SDK durante la compilación

2.3. Descripción de los Archivos de Código

2.3.1. Archivo `lampara.gatt` – Servicio BLE personalizado

A diferencia de la Práctica 15 que usaba el servicio estándar *Environmental Sensing*, en esta práctica se define un **servicio GATT personalizado** con un UUID de 128 bits (12345678-1234-5678-1234-56789ABCDEF0). Esto es útil cuando no existe un perfil estándar que se ajuste al caso de uso, como ocurre con un control multimodal de lámpara.

La característica de control (`...DEF1`) tiene los permisos `READ | WRITE | DYNAMIC`, lo que permite al cliente tanto consultar el estado actual del bombillo como enviar comandos de cambio de modo o configuración.

```

1 PRIMARY_SERVICE, GAP_SERVICE
2 CHARACTERISTIC, GAP_DEVICE_NAME, READ, "PicoW-Lampara"
3
4 PRIMARY_SERVICE, GATT_SERVICE
5
6 // Servicio personalizado de la lampara
7 PRIMARY_SERVICE, 12345678-1234-5678-1234-56789ABCDEF0
8 // Caracteristica de control multimodo
9 CHARACTERISTIC, 12345678-1234-5678-1234-56789ABCDEF1,
10    READ | WRITE | DYNAMIC

```

2.3.2. Archivo `btstack_config.h` – Configuración de BTstack

La configuración utilizada habilita únicamente el rol `LE_PERIPHERAL` (periférico), ya que la Pico W actúa como servidor que es escaneado y conectado por la aplicación móvil. Se permite una sola conexión simultánea (`MAX_NR_HCI_CONNECTIONS = 1`), un buffer ACL de 256 bytes y se deshabilitan los clientes GATT (`MAX_NR_GATT_CLIENTS = 0`).

Se activa también la **extensión de longitud de datos** (`ENABLE_LE_DATA_LENGTH_EXTENSION`), que permite paquetes BLE de hasta 251 bytes en lugar de los 27 bytes del estándar

4.0, lo cual mejora notablemente el rendimiento cuando se transmiten comandos largos como los del temporizador.

2.3.3. Archivo `main.c` – Lógica principal del firmware

El archivo contiene todo el firmware del servidor BLE y se organiza en seis bloques lógicos:

- **Constantes y variables globales:** pines, umbrales de la LDR (`UMBRAL_ENCENDER = 290`, `UMBRAL_APAGAR = 500`) calibrados experimentalmente, enumerado `modo_lampara_t` con los tres modos posibles, y el manejador de conexión.
- **`hardware_init()`:** inicializa el GPIO del relé en su estado de reposo (apagado según la lógica de `RELAY_ACTIVE_LOW`), el LED externo, y el ADC apuntando al canal 0 (GP26) donde está la LDR.
- **`set_lamp()`:** función centralizada de cambio de estado que actualiza simultáneamente el relé, el LED externo y el LED del módulo CYW43, garantizando coherencia visual del sistema.
- **`timer_handler()`:** corazón del sistema. Se ejecuta cada segundo. Lee la LDR, imprime el estado actual al puerto serial y aplica la lógica correspondiente al modo actual: decrementa el temporizador en modo `TIMER`, o compara con los umbrales en modo `AUTO`.
- **`att_write_callback()`:** interpreta los comandos textuales recibidos desde la aplicación. El protocolo definido es:
 - `M:0 / M:1` → modo `MANUAL` apagado/encendido
 - `A:` → activa modo `AUTOMÁTICO`
 - `T:NN` → modo `TEMPORIZADOR` con `NN` segundos
- **`packet_handler()`:** maneja los eventos del stack: configura los anuncios cuando el stack está listo, detecta conexiones y desconexiones (en cuyo caso vuelve a modo `MANUAL` y apaga el foco como medida de seguridad).

2.3.4. Histéresis en el modo automático

Un detalle de diseño relevante es el uso de **histéresis** en el modo `AUTO`, implementada mediante dos umbrales distintos (290 para encender, 500 para apagar). Esto evita el efecto *chattering*: si se usara un único umbral, las pequeñas fluctuaciones de la lectura ADC cerca de ese valor causarían encendidos y apagados continuos. Con dos umbrales separados se garantiza que, una vez encendido, el sistema necesita un cambio significativo de iluminación para apagarse, y viceversa.

2.4. Diseño de la Aplicación Móvil en Flutter

La aplicación móvil se desarrolló en Flutter utilizando el paquete `flutter_blue_plus`, que abstrae la API nativa de Android Bluetooth en una interfaz reactiva basada en Streams.

2.4.1. Estructura de la aplicación

La aplicación se compone de tres pantallas principales:

1. **Pantalla de escaneo:** muestra una lista de dispositivos BLE cercanos filtrados por nombre (PicoW-Lampara). Al pulsar sobre un dispositivo, la app intenta la conexión.
2. **Pantalla principal de control:** contiene tres tarjetas, una por modo. Cada tarjeta envía el comando correspondiente a la característica BLE al ser interactuada:
 - Modo Manual: dos botones grandes *Encender* y *Apagar*.
 - Modo Temporizador: un campo numérico, un botón *Iniciar* y un display de cuenta regresiva sincronizado mediante un `Timer.periodic`.
 - Modo Automático: un `Switch` que activa/desactiva el modo enviando A:.
3. **Indicador de estado:** un círculo coloreado que indica el estado del bombillo (verde = encendido, gris = apagado), actualizado mediante lecturas periódicas de la característica.

2.4.2. Comunicación BLE en Flutter

El flujo de comunicación es el siguiente:

```

1 // 1. Escanear dispositivos
2 FlutterBluePlus.startScan(timeout: Duration(seconds: 10));
3
4 // 2. Conectar al dispositivo
5 await device.connect();
6
7 // 3. Descubrir servicios
8 List<BluetoothService> services = await device.
  discoverServices();
9 final service = services.firstWhere(
10   (s) => s.uuid.toString() == "12345678-1234-5678-1234-56789
    abcdef0");
11 final characteristic = service.characteristics.firstWhere(
12   (c) => c.uuid.toString() == "12345678-1234-5678-1234-56789
    abcdef1");
13
14 // 4. Escribir comando (ej. encender en modo manual)
15 await characteristic.write(utf8.encode("M:1"));

```


2.5. Proceso de Compilación

Con todos los archivos en la carpeta del proyecto, la compilación se realiza desde la terminal:

```

1 cd "C:\Users\soysa\OneDrive\Documentos\Practica 16"
2 mkdir build
3 cd build
4 cmake .. -G "Ninja" -DPICO_SDK_PATH=C:/pico-sdk -DPICO_BOARD=
    pico_w ^
5 -DCMAKE_C_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
    gcc.exe ^
6 -DCMAKE_CXX_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
    g++.exe ^
7 -DCMAKE_ASM_COMPILER=C:/arm-toolchain/.../bin/arm-none-eabi-
    gcc.exe
8 ninja

```

Al finalizar correctamente se genera el archivo `lampara_ble.uf2`, que se flashea en la Raspberry Pi Pico W manteniendo presionado **BOOTSEL** mientras se conecta el cable USB.

2.6. Montaje y Pruebas

Tras flashear el firmware, conectar la aplicación móvil y observar el funcionamiento en el monitor serial (115200 baudios), el sistema produce la siguiente salida:

```

1 =====
2     LAMPARA INTELIGENTE PICO W V2
3 =====
4 Inicializando sistemas...
5 BLE encendido y transmitiendo...
6
7 [INFO] LDR:    840 (0.42V) | Modo: MANUAL          | Foco: OFF
8 [INFO] LDR:    838 (0.42V) | Modo: MANUAL          | Foco: OFF
9 Celular conectado con exito!
10
11 -> Comando recibido de la app: M:1
12 ACCION -> Lampara: ENCENDIDA
13 [INFO] LDR:    840 (0.42V) | Modo: MANUAL          | Foco: ON
14
15 -> Comando recibido de la app: A:
16 Cambiando a Modo AUTOMATICO. Dejando que la LDR decida...
17 [INFO] LDR:    285 (0.14V) | Modo: AUTOMATICO      | Foco: ON
18 Valor LDR bajo de 290. Encendiendo...
19
20 -> Comando recibido de la app: T:10
21 Cambiando a Modo TEMPORIZADOR. Cuenta regresiva: 11 segundos

```

```
22 [INFO] LDR:    840 (0.42V) | Modo: TEMPORIZADOR | Foco: ON
23 ...
24 [INFO] LDR:    841 (0.42V) | Modo: TEMPORIZADOR | Foco: ON
25 Pum! Temporizador terminado. Apagando todo.
26 ACCION -> Lampara: APAGADA
```

Las pruebas confirmaron que:

- El cambio entre modos desde la aplicación es instantáneo.
- La histéresis impide oscilaciones en el modo automático.
- El temporizador apaga el foco con precisión de ± 1 segundo.
- Al desconectar el celular, el foco se apaga automáticamente y vuelve al modo MANUAL como medida de seguridad.

2.7. Complicaciones y Soluciones

Cuadro 3: Errores encontrados durante el desarrollo y sus soluciones

Error Encontrado	Solución Aplicada
El relé no activaba al conectarlo directamente al GPIO de la Pico W.	Se intercaló el transistor 2N2222A con resistencia de base de $1\text{ k}\Omega$ para amplificar la corriente. El GPIO solo entrega 12 mA máx y la bobina del relé necesita $\sim 70\text{ mA}$.
El transistor se calentaba excesivamente y se dañó uno durante las pruebas.	Se agregó el diodo 1N4007 en antiparalelo con la bobina (cátodo a $+5\text{ V}$) para absorber el pico inductivo de back-EMF que estaba destruyendo la unión colector-emisor.
En modo automático, el foco oscilaba (parpadeaba) cuando la luz estaba cerca del umbral.	Se implementó histéresis usando dos umbrales separados (290 para encender, 500 para apagar) en lugar de uno solo, eliminando el <i>chattering</i> .
La aplicación Flutter no encontraba el dispositivo BLE durante el escaneo.	Se solicitaron los permisos de <code>BLUETOOTH_SCAN</code> , <code>BLUETOOTH_CONNECT</code> y <code>ACCESS_FINE_LOCATION</code> en el <code>AndroidManifest.xml</code> y se manejaron las peticiones en runtime con el paquete <code>permission_handler</code> .
Los comandos enviados desde Flutter llegaban como bytes binarios, no como texto.	Se aplicó <code>utf8.encode("M:1")</code> en el lado Flutter antes de escribir, y en el firmware se trató el buffer como cadena ASCII terminada en <code>'\0'</code> .
El temporizador apagaba el foco un segundo antes de lo configurado.	Se compensó sumando 1 al valor recibido (<code>tiempo_restante = atoi(&comando[2]) + 1</code>) ya que el timer decreuenta antes de comparar con cero.

3. Preguntas de Repaso

1. ¿Qué función cumple el transistor en el circuito?

El transistor **2N2222A** actúa como un **interruptor electrónico amplificador de corriente** entre el microcontrolador y la bobina del relevador. Configurado en *modo saturación* (operando como switch), permite que la pequeña corriente de base ($\sim 2.6\text{ mA}$

suministrada por el GPIO a través de una resistencia de $1\text{ k}\Omega$) controle una corriente de colector mucho mayor ($\sim 70\text{ mA}$ necesarios por la bobina del relé). Sin esta etapa de amplificación, sería imposible activar el relevador directamente desde un pin de la Pico W, ya que la corriente requerida por la bobina supera ampliamente la capacidad máxima del GPIO. Adicionalmente, el transistor aísla parcialmente el circuito digital del circuito de potencia, protegiendo al microcontrolador de transitorios.

2. ¿Por qué no debe conectarse directamente el relevador a la Raspberry Pi Pico W?

Por dos razones fundamentales relacionadas con los **límites eléctricos del microcontrolador**:

- **Corriente excesiva:** la bobina del relé de 5 V típicamente consume entre 50 y 80 mA , mientras que un pin GPIO de la Raspberry Pi Pico W tiene una corriente máxima absoluta de 12 mA y una corriente recomendada de operación de solo $3\text{--}8\text{ mA}$. Forzar 70 mA a través de un GPIO destruiría el pin de forma inmediata e incluso podría dañar el chip RP2040 completo.
- **Tensión incompatible:** la bobina del relé requiere 5 V para activarse correctamente, pero los pines GPIO de la Pico W operan a 3.3 V . Aunque algunos relés pueden activar a tensiones más bajas, lo harían de forma errática.

Adicionalmente, conectar directamente el relé expondría al microcontrolador al pico inductivo de back-EMF cuando la bobina se desenergice, lo cual sería catastrófico para los circuitos digitales internos.

3. ¿Qué función tiene el diodo conectado al relevador?

El diodo **1N4007** conectado en *antiparalelo* con la bobina del relé (cátodo al positivo de 5 V , ánodo al colector del transistor) cumple la función de **diodo flyback** o *freewheeling*. Cuando el transistor se apaga, el campo magnético almacenado en la bobina colapsa súbitamente y, según la ley de Lenz, induce un pico de tensión inverso que puede alcanzar varios cientos de voltios:

$$V_{induced} = -L \frac{dI}{dt}$$

Sin protección, este pico destruiría inmediatamente el transistor (cuya tensión V_{CE} máxima es de 40 V). El diodo proporciona un **camino de baja impedancia** por donde la energía almacenada se disipa, recirculando la corriente residual a través de la bobina hasta que se extingue de forma segura. Es un componente **indispensable** en cualquier circuito que conmute cargas inductivas (relés, motores, electroválvulas).

4. ¿Qué es una fotorresistencia?

Una **fotorresistencia o LDR** (*Light Dependent Resistor*) es un componente electrónico pasivo cuya resistencia eléctrica varía en función de la intensidad de la luz que incide sobre su superficie. Está construida con un material **semiconductor fotosensible**, comúnmente **sulfuro de cadmio (CdS)**, depositado en forma de meandro sobre un sustrato cerámico y protegido con una resina transparente.

Su principio de funcionamiento se basa en el **efecto fotoeléctrico interno**: los fotones de la luz incidente, al chocar con los átomos del semiconductor, ceden energía suficiente para liberar electrones de la banda de valencia, aumentando la cantidad de portadores de carga libres y, en consecuencia, la conductividad del material. Es un componente económico, robusto y ampliamente usado en aplicaciones como sensores crepusculares, cámaras automáticas, alumbrado público y sistemas de domótica.

5. ¿Cómo cambia la resistencia de una LDR cuando aumenta la luz?

La resistencia de la LDR **disminuye** cuando aumenta la intensidad luminosa, comportándose de manera **inversamente proporcional**:

- En **oscuridad total**, la resistencia puede superar $1\text{ M}\Omega$ (incluso varios $\text{M}\Omega$).
- Bajo **iluminación ambiente** normal de una habitación, la resistencia es del orden de $5\text{-}20\text{ k}\Omega$.
- Bajo **luz solar directa**, la resistencia cae a unos pocos cientos de ohms ($<1\text{ k}\Omega$).

Este comportamiento se explica físicamente porque al incrementar la luz, más fotones liberan más electrones en el semiconductor, generando más portadores de carga libres y aumentando la conductividad (o equivalentemente, reduciendo la resistencia). En esta práctica, este principio se aprovecha mediante un divisor de tensión con una resistencia fija: el ADC de la Pico W lee una tensión **baja en oscuridad** (LDR de alta resistencia) y una tensión **alta con luz** (LDR de baja resistencia), lo que se traduce en los valores ADC que el firmware usa para tomar decisiones.

6. ¿Qué ventajas ofrece el control inalámbrico mediante Bluetooth?

El control inalámbrico mediante Bluetooth LE aporta múltiples ventajas:

- **Eliminación del cableado**: no se requiere tender cables físicos entre el sistema controlador y la lámpara, reduciendo costos de instalación y permitiendo el despliegue en lugares de difícil acceso.
- **Movilidad del usuario**: el usuario puede controlar la lámpara desde cualquier punto dentro del rango (hasta 100 m en campo abierto, $\sim 10\text{ m}$ en interiores con paredes), sin necesidad de estar cerca de un interruptor físico.

- **Integración con smartphones:** prácticamente todos los dispositivos móviles modernos incluyen Bluetooth LE de forma nativa, eliminando la necesidad de adquirir hardware adicional.
- **Bajo consumo energético:** BLE consume típicamente entre 10 y 100 veces menos energía que WiFi, lo cual es crítico si el sistema se alimenta por batería.
- **Múltiples modos de operación:** un único dispositivo BLE puede recibir comandos complejos (manual, temporizador, automático) sin requerir interruptores adicionales.
- **Seguridad:** BLE soporta emparejamiento con autenticación y cifrado AES-128, dificultando que dispositivos no autorizados controlen la lámpara.
- **Escalabilidad:** es posible añadir más lámparas al sistema sin modificaciones de hardware central, simplemente agregándolas a la lista de dispositivos en la aplicación móvil.

7. ¿Qué diferencia existe entre el modo manual y el modo automático?

La diferencia central radica en **quién toma la decisión** de encender o apagar la lámpara:

- **Modo Manual:** el **usuario** decide directamente el estado de la lámpara mediante los botones de la aplicación. El firmware se limita a ejecutar el comando recibido (M:0 o M:1) sin tomar decisiones autónomas. La LDR se sigue leyendo y mostrando en el monitor serial, pero su valor no influye en el estado del foco. Este modo se usa cuando se desea control total y predecible.
- **Modo Automático:** el **firmware** decide autónomamente el estado de la lámpara según la lectura de la LDR. Si el valor ADC cae por debajo del umbral inferior (290, indicando oscuridad), enciende la lámpara; si supera el umbral superior (500, indicando luz suficiente), la apaga. El usuario solo activa o desactiva este modo, pero no controla directamente el foco. Este modo emula el comportamiento del alumbrado público automático y es útil para ahorro de energía.

Adicionalmente, el modo Temporizador es un híbrido: el usuario decide el tiempo de encendido, pero el firmware ejecuta autónomamente el apagado al finalizar la cuenta regresiva.

8. ¿Qué ventajas tiene Flutter para el desarrollo de aplicaciones móviles?

Flutter, el framework desarrollado por Google, ofrece varias ventajas significativas para el desarrollo móvil:

- **Multiplataforma con un solo código base:** permite compilar la misma aplicación para Android, iOS, Web, Windows, macOS y Linux a partir de un único proyecto en Dart, reduciendo drásticamente el tiempo y costo de desarrollo comparado con escribir aplicaciones nativas separadas.
- **Rendimiento nativo:** Flutter compila a código máquina ARM nativo, no usa un *bridge* JavaScript como React Native, lo cual se traduce en interfaces fluidas a 60-120 fps.
- **Hot Reload:** permite ver los cambios en la aplicación en menos de un segundo sin reiniciarla, acelerando enormemente el ciclo de desarrollo y pruebas.
- **Sistema de widgets compositivo:** la interfaz se construye declarativamente combinando widgets pequeños, lo cual facilita la reutilización de componentes y la separación de responsabilidades.
- **Ecosistema de paquetes:** pub.dev ofrece miles de paquetes mantenidos por la comunidad, incluyendo `flutter_blue_plus` usado en esta práctica para BLE.
- **Diseño visual consistente:** Flutter implementa Material Design (Android) y Cupertino (iOS) de forma nativa, ofreciendo apariencia idiomática en cada plataforma.
- **Backed por Google:** la empresa lo respalda activamente, lo cual garantiza mantenimiento a largo plazo y actualizaciones frecuentes.

9. ¿Qué ocurre si el relevador no utiliza transistor de activación?

Las consecuencias serían severas tanto a nivel eléctrico como funcional:

1. **Daño inmediato al GPIO:** al intentar suministrar los 70 mA que requiere la bobina del relé, el pin GPIO de la Pico W (con un límite absoluto de 12 mA) se sobrecalienta y los transistores internos de salida se destruyen. El pin queda permanentemente inutilizable o, en el peor caso, el chip RP2040 entero se daña.
2. **Caída de tensión severa:** la fuente de alimentación de 3.3 V interna de la Pico W no puede entregar esa cantidad de corriente, provocando que el voltaje caiga drásticamente, lo que puede causar reinicios espontáneos del microcontrolador y comportamiento errático.
3. **El relé probablemente no activaría:** aun si el GPIO sobreviviera momentáneamente, la tensión disponible (3.3 V con caídas) no sería suficiente para superar el umbral de activación de la bobina (típicamente 4 V para un relé de 5 V), por lo que la lámpara nunca encendería.
4. **Pico inductivo directo al GPIO:** al desenergizarse la bobina sin diodo de protección, el pico de back-EMF entraría directamente al pin GPIO, casi con certeza dañando los diodos ESD de protección internos del RP2040.

5. **Ruido eléctrico:** el cambio brusco de corriente generaría EMI (interferencia electromagnética) que podría afectar la comunicación BLE y la lectura ADC.

En resumen: sin el transistor, el sistema no funcionaría y casi con certeza destruiría la Pico W en el primer intento.

10. ¿Qué función cumple el temporizador dentro del sistema?

El temporizador permite el **encendido programado por tiempo limitado** de la lámpara, ofreciendo varias funciones prácticas:

- **Ahorro energético automatizado:** la lámpara se enciende solamente durante el tiempo necesario y se apaga sola, evitando el desperdicio de energía por olvidos del usuario.
- **Comodidad y seguridad:** útil para iluminar áreas durante un tiempo controlado (por ejemplo, para que una persona se duerma, para iluminar un patio durante una caminata nocturna, o para simular presencia durante una ausencia).
- **Lógica desacoplada del usuario:** una vez configurado, el usuario no necesita recordar apagar la lámpara, el sistema lo hace autónomamente.
- **Verificación de funcionamiento:** permite probar fácilmente el sistema desde la aplicación sin tener que enviar dos comandos separados.
- **Implementación técnica:** en el firmware, se implementa mediante una variable `tiempo_restante` que se decrementa en cada iteración del timer del run loop de BTstack (cada 1000 ms). Cuando llega a cero, se llama a `set_lamp(false)` y se restablece el modo a MANUAL.

Esta función es la base de aplicaciones más sofisticadas como las luces nocturnas para niños con apagado automático tras un período de tiempo, o los sistemas de iluminación de escaleras que se apagan tras detectar movimiento durante un tiempo configurado.

4. Conclusiones

La práctica permitió integrar de forma exitosa múltiples conceptos vistos a lo largo del semestre: comunicación inalámbrica mediante Bluetooth Low Energy, lectura analógica con el ADC del RP2040, control de cargas mediante etapa de potencia con transistor, protección contra picos inductivos, y desarrollo de aplicación móvil multi-plataforma. El resultado final es un sistema funcional, robusto y replicable, que emula a pequeña escala el comportamiento de productos comerciales de domótica como las lámparas inteligentes Philips Hue o WiZ.

Uno de los aprendizajes más valiosos fue comprender la **importancia de las etapas de aislamiento y amplificación** entre la lógica digital del microcontrolador y las cargas reales del mundo físico. El transistor y el diodo flyback no son simples *detalles de*

circuito, sino elementos indispensables sin los cuales el sistema directamente no funcionaría y el microcontrolador se destruiría. Este patrón de diseño se repite en cualquier aplicación de electrónica de potencia y es transversal a toda la industria embebida.

La implementación de la **histéresis** en el modo automático fue otro aprendizaje técnico destacable: una solución elegante y de costo cero a un problema (*chattering*) que de otro modo haría al sistema impráctico. Refleja un principio general del diseño de sistemas de control: nunca confiar en un único umbral cuando la señal de entrada tiene ruido.

El desarrollo de la aplicación Flutter introdujo el paradigma de **programación reactiva basada en streams**, un modelo que se ha convertido en estándar en el desarrollo móvil moderno. La integración con el firmware se logró gracias al diseño del protocolo de comandos textuales (M:1, T:30, A:), que aunque simple, resultó suficientemente expresivo para todos los casos de uso requeridos.

Posibles mejoras:

- **Notificaciones BLE en lugar de polling:** en la implementación actual la aplicación lee periódicamente el estado del foco. Cambiar a notificaciones permitiría una actualización instantánea cuando el estado cambia por decisión autónoma del firmware (modo AUTO o TIMER).
- **Persistencia del último modo:** guardar el modo y configuración en la flash del RP2040 para que sobreviva reinicios del sistema.
- **Múltiples lámparas:** ampliar la aplicación para que pueda controlar varias Pico W simultáneamente, formando una red de domótica más completa.
- **Sensor PIR:** agregar un sensor de movimiento infrarrojo pasivo (PIR) para implementar un cuarto modo: encendido por detección de presencia.
- **Dimming PWM:** en lugar de encendido binario, controlar la intensidad mediante PWM aplicado a un MOSFET (requeriría reemplazar el relé por un circuito de control de fase con TRIAC para CA).
- **Seguridad:** implementar emparejamiento BLE con clave para evitar que cualquier dispositivo cercano pueda controlar la lámpara.
- **Programación horaria:** agregar a Flutter un calendario que envíe comandos de encendido/apagado en horarios programados, transformando el sistema en un timer semanal completo.
- **Integración WiFi/MQTT:** aprovechar el doble radio del CYW43 para enviar el estado de la lámpara a un servidor MQTT, permitiendo control desde Internet y registro histórico.

Anexos

Anexo A – Archivo `lampara.gatt`

```

1 PRIMARY_SERVICE , GAP_SERVICE
2 CHARACTERISTIC , GAP_DEVICE_NAME , READ , "PicoW-Lampara"
3
4 PRIMARY_SERVICE , GATT_SERVICE
5
6 // Servicio de la lampara
7 // UUID: 12345678-1234-5678-1234-56789ABCDEF0
8 PRIMARY_SERVICE , 12345678-1234-5678-1234-56789ABCDEF0
9
10 // Caracteristica de control ON/OFF, AUTO y TIMER
11 // UUID: 12345678-1234-5678-1234-56789ABCDEF1
12 CHARACTERISTIC , 12345678-1234-5678-1234-56789ABCDEF1 ,
13     READ | WRITE | DYNAMIC

```

Anexo B – Archivo btstack_config.h

```

1 #ifndef BTSTACK_CONFIG_H
2 #define BTSTACK_CONFIG_H
3
4 #define ENABLE_LE_PERIPHERAL
5 #define ENABLE_LOG_INFO
6 #define ENABLE_LOG_ERROR
7 #define ENABLE_PRINTF_HEXDUMP
8 #define ENABLE_LE_DATA_LENGTH_EXTENSION
9
10 #define HCI_ACL_PAYLOAD_SIZE 256
11 #define HCI_OUTGOING_PRE_BUFFER_SIZE 4
12 #define HCI_ACL_CHUNK_SIZE_ALIGNMENT 4
13
14 #define MAX_NR_BTSTACK_LINK_KEY_DB_MEMORY_ENTRIES 0
15 #define MAX_NR_HCI_CONNECTIONS 1
16 #define MAX_NR_L2CAP_SERVICES 3
17 #define MAX_NR_L2CAP_CHANNELS 3
18 #define MAX_NR_GATT_CLIENTS 0
19 #define MAX_NR_SM_LOOKUP_ENTRIES 3
20 #define MAX_NR_WHITELIST_ENTRIES 1
21 #define MAX_NR_LE_DEVICE_DB_ENTRIES 4
22 #define NVM_NUM_DEVICE_DB_ENTRIES 4
23
24 #define MAX_ATT_DB_SIZE 512
25
26 #endif

```

Anexo C – Archivo main.c

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include "pico/stdlib.h"
5 #include "hardware/adc.h"
6 #include "pico/cyw43_arch.h"
7 #include "pico/btstack_cyw43.h"
8
9 #include "btstack_run_loop.h"
10 #include "btstack_event.h"
11 #include "ble/att_server.h"
12 #include "ble/sm.h"
13 #include "gap.h"
14 #include "hci.h"
15 #include "l2cap.h"
16
17 #include "lampara.h"
18
19 // PINES Y CONFIGURACION
20 #define RELAY_GPIO 15
21 #define RELAY_ACTIVE_LOW 1
22 #define LED_EXT_GPIO 2
23
24 #define PIN_LDR 26
25 #define ADC_CANAL 0
26
27 #define UMBRAL_ENCENDER 290
28 #define UMBRAL_APAGAR 500
29
30 // ESTADOS Y MODOS
31 typedef enum {
32     MODO_MANUAL,
33     MODO_AUTO,
34     MODO_TIMER
35 } modo_lampara_t;
36
37 static const char* nombres_modos[] = {"MANUAL", "AUTOMATICO",
    "TEMPORIZADOR"};
38
39 static modo_lampara_t modo_actual = MODO_MANUAL;
40 static bool lamp_state = false;
41 static int32_t tiempo_restante = 0;
42
43 static hci_con_handle_t con_handle = HCI_CON_HANDLE_INVALID;
```

```
44 static btstack_packet_callback_registration_t
    hci_event_callback_registration;
45 static btstack_timer_source_t loop_timer;
46
47 // HARDWARE
48 static void hardware_init(void) {
49     gpio_init(RELAY_GPIO);
50     gpio_set_dir(RELAY_GPIO, GPIO_OUT);
51     gpio_put(RELAY_GPIO, RELAY_ACTIVE_LOW ? 1 : 0);
52
53     gpio_init(LED_EXT_GPIO);
54     gpio_set_dir(LED_EXT_GPIO, GPIO_OUT);
55     gpio_put(LED_EXT_GPIO, 0);
56
57     adc_init();
58     adc_gpio_init(PIN_LDR);
59     adc_select_input(ADC_CANAL);
60 }
61
62 static void set_lamp(bool state) {
63     if (lamp_state == state) return;
64
65     lamp_state = state;
66     gpio_put(RELAY_GPIO, RELAY_ACTIVE_LOW ? !state : state);
67     gpio_put(LED_EXT_GPIO, state);
68     cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, state);
69     printf("ACCION -> Lampara: %s\n", state ? "ENCENDIDA" : "
        APAGADA");
70 }
71
72 // TIMER PRINCIPAL
73 static void timer_handler(btstack_timer_source_t *ts) {
74     btstack_run_loop_set_timer(ts, 1000);
75     btstack_run_loop_add_timer(ts);
76
77     uint16_t valor_adc = adc_read();
78     float voltaje = valor_adc * 3.3f / 65535.0f;
79
80     printf("[INFO] LDR: %5d (%.2fV) | Modo: %-12s | Foco: %s\n
        ",
81         valor_adc, voltaje, nombres_modos[modo_actual],
82         lamp_state ? "ON" : "OFF");
83
84     if (modo_actual == MODO_TIMER) {
85         if (tiempo_restante > 0) {
86             tiempo_restante--;
87             if (tiempo_restante <= 0) {
```

```

88         printf("Pum! Temporizador terminado. Apagando
           todo.\n");
89         set_lamp(false);
90         modo_actual = MODO_MANUAL;
91     }
92 }
93 } else if (modo_actual == MODO_AUTO) {
94     if (valor_adc < UMBRAL_ENCENDER && !lamp_state) {
95         printf("Valor LDR bajo de %d. Encendiendo...\n",
           UMBRAL_ENCENDER);
96         set_lamp(true);
97     } else if (valor_adc > UMBRAL_APAGAR && lamp_state) {
98         printf("Valor LDR subio de %d. Apagando...\n",
           UMBRAL_APAGAR);
99         set_lamp(false);
100     }
101 }
102 }
103
104 // BLE
105 static const uint8_t adv_data[] = {
106     0x02, 0x01, 0x06,
107     0x0E, 0x09, 'P','i','c','o','W','-','L','a','m','p','a','r',
           ',','a',
108 };
109 static const uint8_t adv_data_len = sizeof(adv_data);
110
111 static uint16_t att_read_callback(hci_con_handle_t
   connection_handle,
112     uint16_t att_handle, uint16_t offset, uint8_t *buffer,
           uint16_t buffer_size) {
113     UNUSED(connection_handle);
114     if (att_handle ==
           ATT_CHARACTERISTIC_12345678_1234_5678_1234_56789ABCDEF1_01_VALUE_HANDLE) {
115         if (buffer) buffer[0] = lamp_state ? '1' : '0';
116         return 1;
117     }
118     return 0;
119 }
120
121 static int att_write_callback(hci_con_handle_t
   connection_handle,
122     uint16_t att_handle, uint16_t transaction_mode, uint16_t
           offset,
123     uint8_t *buffer, uint16_t buffer_size) {
124     UNUSED(connection_handle);
125     UNUSED(transaction_mode);

```

```

126     UNUSED(offset);
127
128     if (att_handle ==
129         ATT_CHARACTERISTIC_12345678_1234_5678_1234_56789ABCDEF1_01_VALUE_HANDLE) {
130
131         && buffer_size > 0) {
132             char comando[20];
133             uint16_t len = buffer_size < 19 ? buffer_size : 19;
134             memcpy(comando, buffer, len);
135             comando[len] = '\0';
136
137             printf("\n-> Comando recibido de la app: %s\n",
138                 comando);
139
140             if (comando[0] == 'M' && comando[1] == ':') {
141                 modo_actual = MODO_MANUAL;
142                 set_lamp(comando[2] == '1');
143             } else if (comando[0] == 'A' && comando[1] == ':') {
144                 modo_actual = MODO_AUTO;
145             } else if (comando[0] == 'T' && comando[1] == ':') {
146                 modo_actual = MODO_TIMER;
147                 tiempo_restante = atoi(&comando[2]) + 1;
148                 set_lamp(true);
149             }
150         }
151     }
152     return 0;
153 }
154
155 static void packet_handler(uint8_t packet_type, uint16_t
156     channel,
157     uint8_t *packet, uint16_t size) {
158     UNUSED(channel);
159     UNUSED(size);
160     if (packet_type != HCI_EVENT_PACKET) return;
161
162     switch (hci_event_packet_get_type(packet)) {
163     case BTSTACK_EVENT_STATE:
164         if (btstack_event_state_get_state(packet) ==
165             HCI_STATE_WORKING) {
166             bd_addr_t null_addr;
167             memset(null_addr, 0, 6);
168             gap_advertisements_set_params(0x0030, 0x00a0,
169                 0, 0,
170                 null_addr, 0x07,
171                 0x00);
172             gap_advertisements_set_data(adv_data_len, (
173                 uint8_t *)adv_data);
174             gap_advertisements_enable(1);
175         }
176     }
177 }

```

```

166         printf("BLE encendido y transmitiendo...\n");
167     }
168     break;
169     case HCI_EVENT_DISCONNECTION_COMPLETE:
170         con_handle = HCI_CON_HANDLE_INVALID;
171         set_lamp(false);
172         modo_actual = MODO_MANUAL;
173         gap_advertisements_enable(1);
174         printf("\nCelular desconectado. Foco apagado.\n");
175         break;
176     case HCI_EVENT_LE_META:
177         if (hci_event_le_meta_get_subevent_code(packet)
178             == HCI_SUBEVENT_LE_CONNECTION_COMPLETE) {
179             con_handle =
180                 hci_subevent_le_connection_complete_get_connection_handle
181                 (packet);
182             printf("\nCelular conectado con exito!\n");
183         }
184         break;
185     }
186 }
187
188 int main(void) {
189     stdio_init_all();
190
191     if (cyw43_arch_init()) {
192         printf("Error: cyw43 init failed.\n");
193         return -1;
194     }
195
196     hardware_init();
197
198     l2cap_init();
199     sm_init();
200     att_server_init(profile_data, att_read_callback,
201                     att_write_callback);
202
203     hci_event_callback_registration.callback = &packet_handler
204     ;
205     hci_add_event_handler(&hci_event_callback_registration);
206     att_server_register_packet_handler(packet_handler);
207
208     hci_power_control(HCI_POWER_ON);
209
210     btstack_run_loop_set_timer_handler(&loop_timer,
211                                         timer_handler);
212     btstack_run_loop_set_timer(&loop_timer, 1000);
213     btstack_run_loop_add_timer(&loop_timer);

```

```
209
210     printf("\nLAMPARA INTELIGENTE PICO W V2\n");
211     printf("Iniciando sistemas...\n\n");
212
213     btstack_run_loop_execute();
214
215     return 0;
216 }
```

Anexo D – Archivo CMakeLists.txt

```
1  cmake_minimum_required(VERSION 3.12)
2
3  set(PICO_BOARD pico_w CACHE STRING "Board type")
4  include(pico_sdk_import.cmake)
5
6  if (PICO_SDK_VERSION_STRING VERSION_LESS "1.5.0")
7      message(FATAL_ERROR "Raspberry Pi Pico SDK version 1.5.0 (
8          or later) required.")
9  endif()
10
11 project(lampara_ble C CXX ASM)
12 set(CMAKE_C_STANDARD 11)
13 set(CMAKE_CXX_STANDARD 17)
14
15 pico_sdk_init()
16
17 add_executable(lampara_ble main.c)
18
19 target_link_libraries(lampara_ble
20     pico_stdlib
21     hardware_adc
22     pico_btstack_ble
23     pico_btstack_cyw43
24     pico_cyw43_arch_none
25 )
26
27 target_include_directories(lampara_ble PRIVATE
28     ${CMAKE_CURRENT_LIST_DIR}
29 )
30
31 pico_btstack_make_gatt_header(lampara_ble PRIVATE
32     "${CMAKE_CURRENT_LIST_DIR}/lampara.gatt"
33 )
34
35 pico_enable_stdio_usb(lampara_ble 1)
```



```
35 pico_enable_stdio_uart(lampara_ble 0)
36
37 pico_add_extra_outputs(lampara_ble)
```

Anexo E – Fragmento del código Flutter (Pantalla de control)

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_blue_plus/flutter_blue_plus.dart';
3  import 'dart:convert';
4  import 'dart:async';
5
6  class ControlScreen extends StatefulWidget {
7    final BluetoothCharacteristic characteristic;
8    const ControlScreen({Key? key, required this.characteristic
9      }) : super(key: key);
10
11    @override
12    State<ControlScreen> createState() => _ControlScreenState();
13  }
14
15  class _ControlScreenState extends State<ControlScreen> {
16    bool _autoMode = false;
17    int _timerSeconds = 30;
18    int _remaining = 0;
19    Timer? _countdown;
20
21    Future<void> _sendCommand(String cmd) async {
22      await widget.characteristic.write(utf8.encode(cmd));
23    }
24
25    void _startTimer() {
26      _sendCommand("T:${_timerSeconds}");
27      setState(() => _remaining = _timerSeconds);
28      _countdown?.cancel();
29      _countdown = Timer.periodic(const Duration(seconds: 1), (t) {
30        if (_remaining <= 0) {
31          t.cancel();
32        } else {
33          setState(() => _remaining--);
34        }
35      });
36    }
37
38    @override
39    Widget build(BuildContext context) {
```

```
39     return Scaffold(  
40       appBar: AppBar(title: const Text("Control de Lampara")),  
41       body: ListView(  
42         padding: const EdgeInsets.all(16),  
43         children: [  
44           // Modo Manual  
45           Card(  
46             child: Padding(  
47               padding: const EdgeInsets.all(12),  
48               child: Column(  
49                 children: [  
50                   const Text("Modo Manual", style: TextStyle(  
51                     fontSize: 20)),  
52                   Row(  
53                     mainAxisAlignment: MainAxisAlignment.  
54                       spaceEvenly,  
55                     children: [  
56                       ElevatedButton.icon(  
57                         icon: const Icon(Icons.lightbulb),  
58                         label: const Text("Encender"),  
59                         onPressed: () => _sendCommand("M:1"),  
60                       ),  
61                       ElevatedButton.icon(  
62                         icon: const Icon(Icons.  
63                           lightbulb_outline),  
64                         label: const Text("Apagar"),  
65                         onPressed: () => _sendCommand("M:0"),  
66                       ),  
67                     ],  
68                   ),  
69                 ],  
70               ),  
71             ),  
72           // Modo Automatico  
73           Card(  
74             child: SwitchListTile(  
75               title: const Text("Modo Automatico (LDR)"),  
76               value: _autoMode,  
77               onChanged: (v) {  
78                 setState(() => _autoMode = v);  
79                 if (v) _sendCommand("A:");  
80                 else _sendCommand("M:0");  
81               },  
82             ),  
83           // Temporizador  
84           Card(  
85             child: SwitchListTile(  
86               title: const Text("Modo Temporizador (Timer)"),  
87               value: _timerMode,  
88               onChanged: (v) {  
89                 setState(() => _timerMode = v);  
90                 if (v) _sendCommand("T:");  
91                 else _sendCommand("M:0");  
92               },  
93             ),  
94           ],  
95         ),  
96       ),  
97     ),  
98   ),  
99 ),  
100
```

```
84         child: Padding(  
85           padding: const EdgeInsets.all(12),  
86           child: Column(  
87             children: [  
88               const Text("Temporizador", style: TextStyle(  
89                 fontSize: 20)),  
90               Slider(  
91                 value: _timerSeconds.toDouble(),  
92                 min: 5, max: 300, divisions: 59,  
93                 label: "$_timerSeconds s",  
94                 onChanged: (v) => setState(() =>  
95                   _timerSeconds = v.toInt()),  
96               ),  
97               ElevatedButton(  
98                 onPressed: _startTimer,  
99                 child: Text("Iniciar ($_timerSeconds s)"),  
100               ),  
101               if (_remaining > 0)  
102                 Text("Restante: $_remaining s",  
103                   style: const TextStyle(fontSize: 24))  
104             ],  
105           ),  
106         ],  
107       ),  
108     );  
109   }  
110 }
```

Evidencia Fotográfica

En esta sección se presentan imágenes del montaje del circuito, capturas del monitor serial mostrando el sistema operativo en sus tres modos, y capturas de la aplicación móvil Flutter durante su funcionamiento.

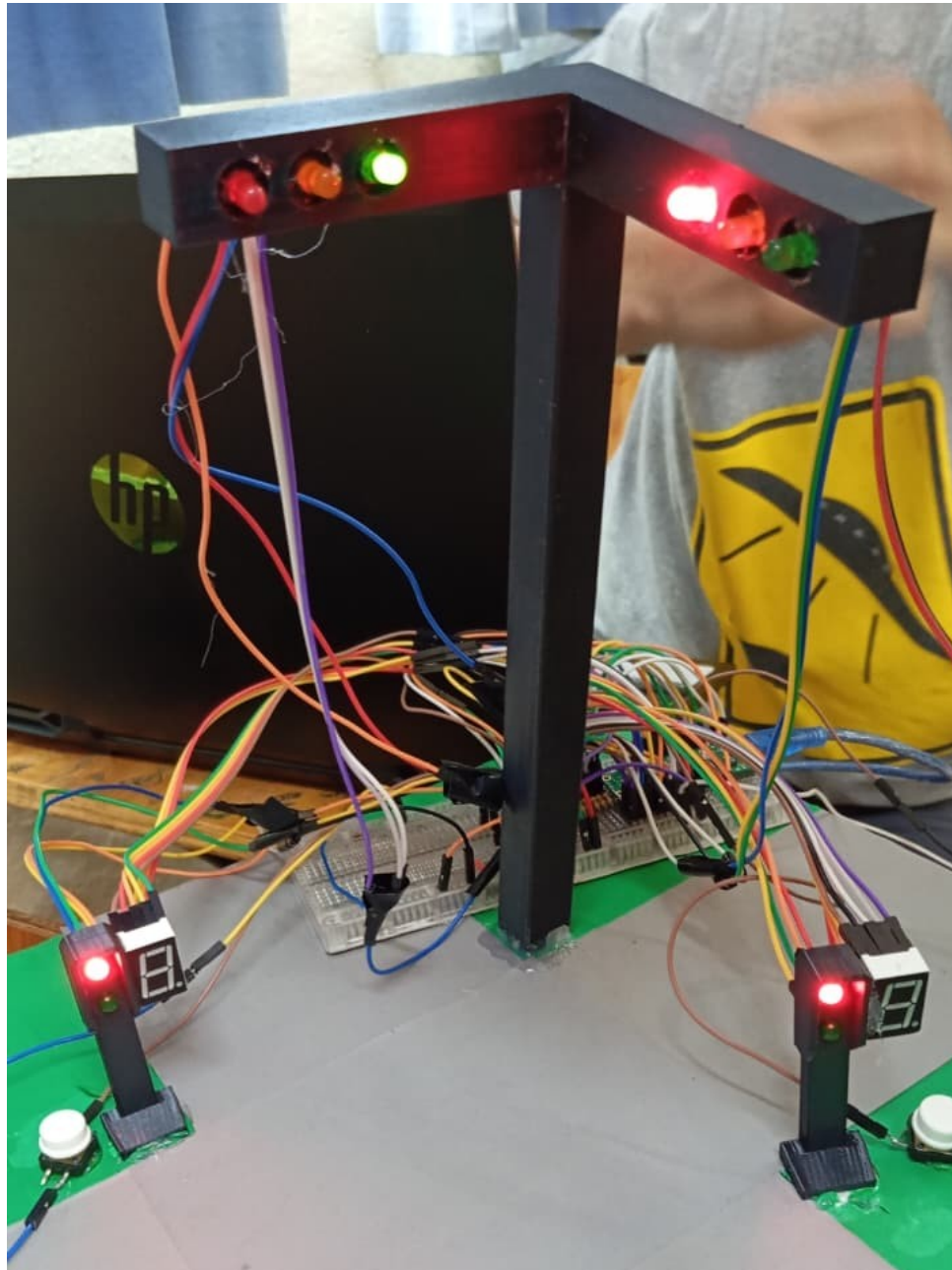


Figura 1: Montaje físico del circuito en protoboard: Raspberry Pi Pico W, transistor 2N2222A, módulo relevador de 5 V, diodo 1N4007, fotorresistencia con divisor de tensión y bombillo de prueba.

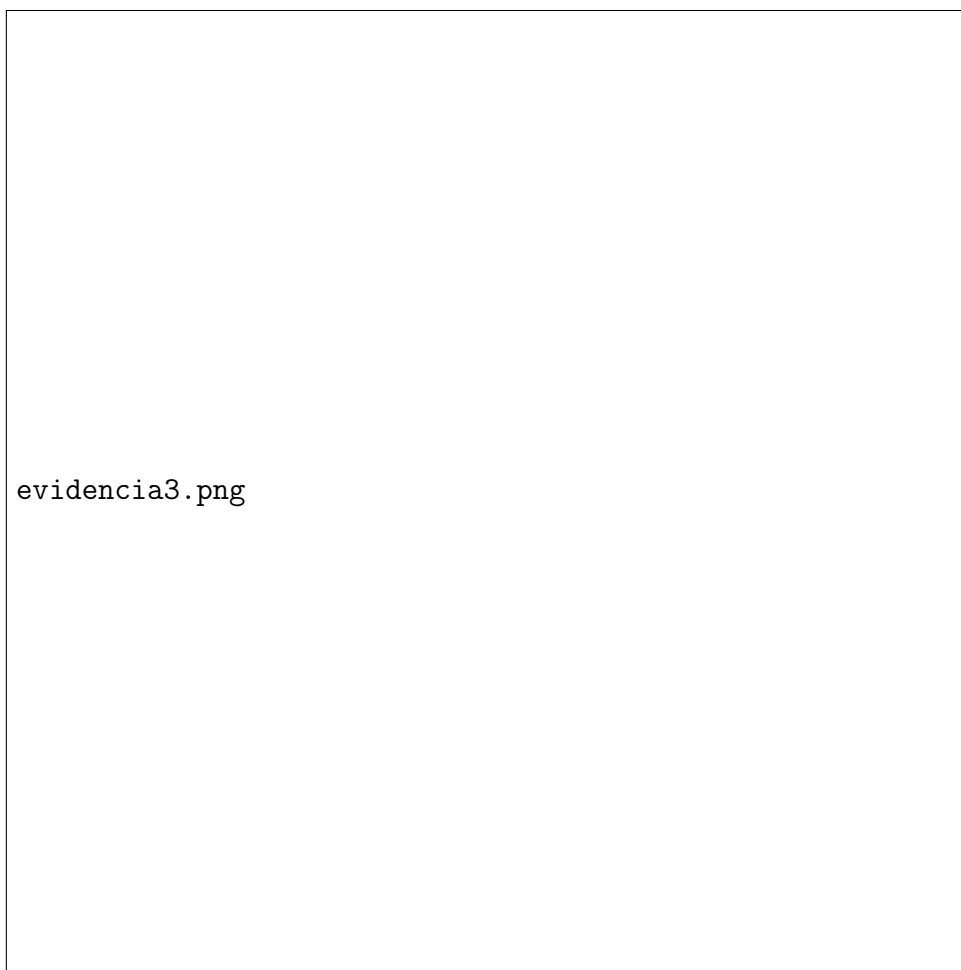


Figura 2: Capturas de la aplicación móvil Flutter mostrando las tres tarjetas de control (modo manual, modo automático y temporizador) durante su uso en un dispositivo Android.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.2). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] BlueKitchen GmbH, *BTstack Manual*. <https://bluekitchen-gmbh.com/btstack/>, 2024.
- [4] Charles Crete, *flutter_blue_plus – Flutter plugin for Bluetooth LE*. https://pub.dev/packages/flutter_blue_plus, 2024.
- [5] Google LLC, *Flutter Documentation*. <https://docs.flutter.dev/>, 2024.

- [6] ON Semiconductor, *2N2222A NPN General Purpose Amplifier Datasheet*. <https://www.onsemi.com/pdf/datasheet/p2n2222a-d.pdf>, 2013.
- [7] Vishay Semiconductors, *1N4007 Standard Recovery Rectifier Datasheet*. <https://www.vishay.com/docs/88503/1n4001.pdf>, 2020.
- [8] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.